Ideation in Software Design

Module 3 (09)

216U01E701

Software Design Thiniking

July- Dec 2026

Module 3: Ideation in Software Design

- Creativity techniques: brainstorming, SCAMPER, mind mapping
- Generating multiple design alternatives
- Storyboarding and user journey mapping
- Design heuristics for software solutions
- Evaluating idea feasibility and prioritization

Ideation

- Bridges user needs (user stories, personas, pain points) and concrete software concepts (architecture, UI/UX flows, feature sets, technical implementations)
- Transform abstract user problems into high-value, technically sound software concepts before costly development begins.

Objectives of Ideation

- Understand Ideation
- Learn idea generation methods
- Apply ideation to software problems
- Develop innovative software solutions

Why Ideation?

- Encourages creativity
- Solves user problems
- Improves software quality
- Promotes innovation

Characteristics of Good design

- User-centric
- Feasible
- Innovative
- Valuable
- Scalable

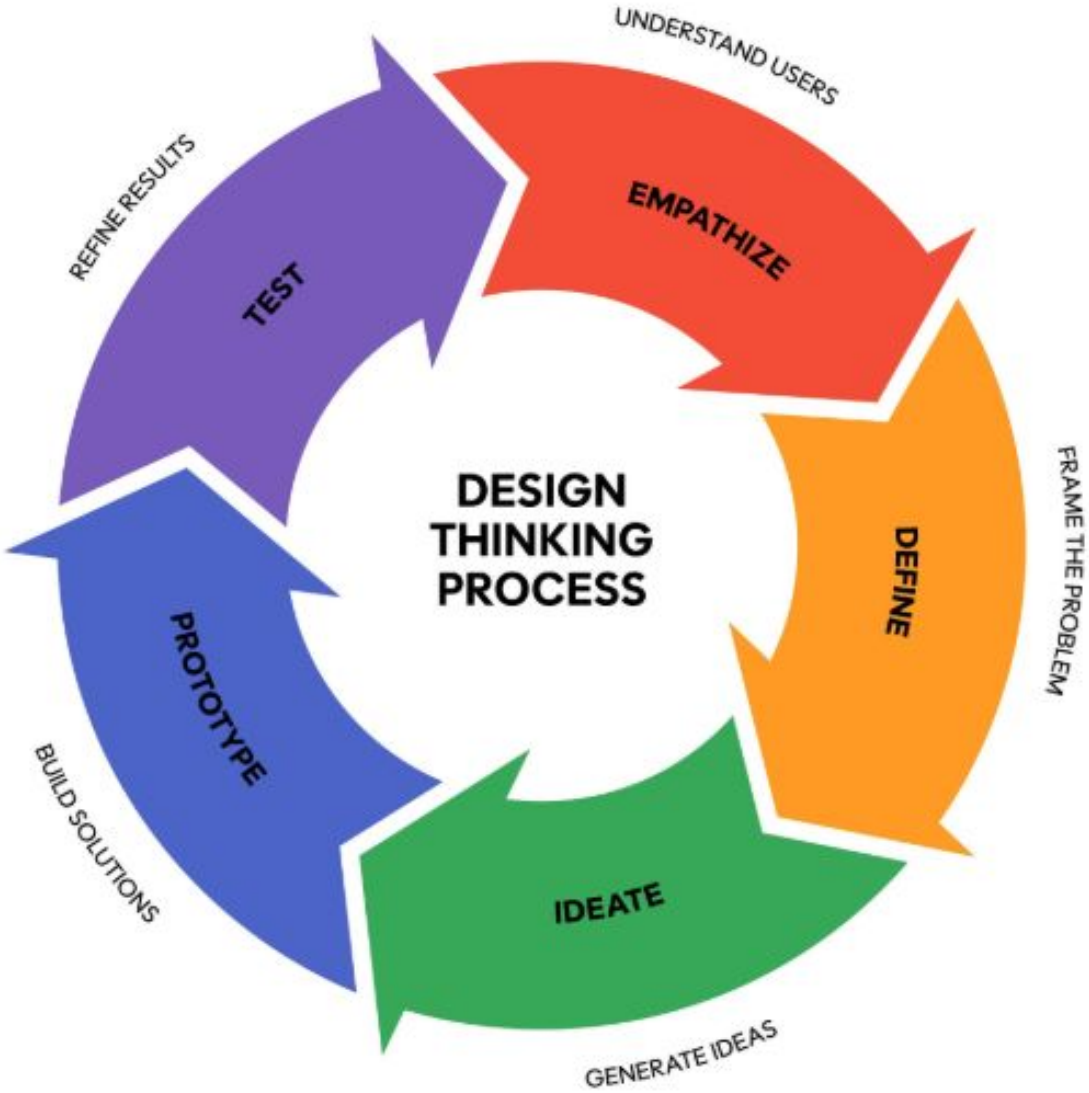
Ideation Process

- Understand problem
- Generate ideas
- Evaluate ideas
- Select best ideas
- Refine ideas

Ideation

- Bridges user needs (user stories, personas, pain points) and concrete software concepts (architecture, UI/UX flows, feature sets, technical implementations)
- Transform abstract user problems into high-value, technically sound software concepts before costly development begins.

Ideation


UNDERSTAND USERS
EMPATHIZE
FRAME THE PROBLEM
DEFINE
GENERATE IDEAS
IDEATE
BUILD SOLUTIONS
PROTOTYPE
REFINE RESULTS
TEST
DESIGN THINKING PROCESS

Ideation

- Brainstorming & Brainwriting
- SCAMPER
- Mind Mapping

Ideation

Brainstorming:

- Traditional group session focused on generating as many ideas as possible in a short window.
- Key rules: defer judgment, encourage wild ideas, build on others' thoughts, and prioritize quantity over initial quality.

Ideation

Brainwriting (6-3-5 Method):

- To prevent dominant voices from hijacking brainstorming, 6 participants write down 3 ideas on a sheet of paper over 5 minutes.
- The sheets are passed to the right, and the next person expands on those ideas. After 6 rounds, 108 ideas are produced in 30 minutes.

Creativity techniques: brainstorming, SCAMPER, mind mapping

- **Brainstorming is a group creativity technique in which participants generate a large number of ideas for a problem without judging or criticizing the ideas during the initial generation phase.**

Main Objective:

- **Generate quantity first, evaluate quality later.**

.

Brainstorming :Key Principles of Brainstorming

1. Defer Judgment

Do not immediately say:

"That idea won't work."

Instead, record it and evaluate it later.

2. Encourage Quantity

The team should generate **many ideas** rather than trying to find one perfect solution immediately.

3. Encourage Wild Ideas

Unusual ideas can lead to practical innovations.

Example:

"What if students could talk to the college ERP like ChatGPT?"

Continue..

4. Build on Others' Ideas

One person's idea can become the starting point for another idea.

Example:

Student A: "Send attendance notifications."

Student B: "Add prediction to tell students if their attendance may fall below 75%.“

5. Keep the Problem Clearly Defined

Brainstorming should focus on a specific **How Might We (HMW)** question.

Example:

How might we help students manage their academic deadlines?

Brainstorming Process

Identify Problem
↓
Define HMW Question
↓
Set Rules
↓
Generate Ideas
↓
Record Ideas
↓
Group Similar Ideas
↓
Evaluate
↓
Select Best Ideas

Hospital Appointment System Problem

Patients spend too much time waiting.

Ideas:

- Online appointment booking
- Digital token
- Live queue tracking
- Doctor availability indicator
- SMS notification
- Estimated waiting time
- Video consultation
- AI symptom pre-screening
- Automatic appointment reminders
- Self-service kiosk

SCAMPER

- An acronym-driven framework used to iterate on an existing product, service, or concept by asking targeted questions:
- **S**ubstitute: What components, materials, or steps can be swapped out?
- **C**ombine: Can two distinct features or services be merged?
- **A**dapt: How can a solution from a different industry be applied here?
- **M**odify / Magnify: What happens if you enlarge, shrink, or exaggerate a feature?
- **P**ut to another use: How might a completely different user group utilize this?
- **E**liminate: What is the most radical element you can remove?
- **R**everse / Rearrange: What if the process order or core mechanics were flipped?

Example of SCAMPER in Software

Substitute

Combine

Adapt

Modify

Put to another use

Eliminate

Reverse

SCAMPER Element	Question	Example for Food Delivery App
Substitute	What can be replaced?	Replace manual customer support with an AI chatbot available 24/7.
Combine	What can be combined?	Combine food delivery with grocery shopping and medicine delivery in one app.
Adapt	What ideas from other industries can be used?	Adapt ride-sharing's live GPS tracking to show the delivery partner's exact location.
Modify	What can be changed or improved?	Introduce voice ordering, larger food images, and personalized restaurant recommendations.
Put to Another Use	Can the app serve another purpose?	Allow users to book catering services or order food for office meetings and events.
Eliminate	What can be removed?	Eliminate lengthy registration by enabling login through Google, Apple, or phone number with OTP.
Reverse (Rearrange)	Can the process be rearranged?	Allow users to schedule food deliveries before selecting the restaurant, or suggest meals based on delivery time and location first.

Existing App

- User searches for a restaurant.
- Selects food items.
- Makes payment.
- Restaurant prepares food.
- Delivery partner delivers the order.

Using SCAMPER

- **Improved App Using SCAMPER**
- **Substitute:** AI chatbot handles customer queries instantly.
- **Combine:** One app supports food, groceries, and pharmacy deliveries.
- **Adapt:** Live GPS tracking with estimated arrival time.
- **Modify:** AI recommends meals based on previous orders and dietary preferences.
- **Put to Another Use:** Companies can place bulk lunch orders for employees.
- **Eliminate:** One-tap reorder for favorite meals, reducing ordering time.
- **Reverse:** Recommend dishes first based on user preferences and budget, then show restaurants offering those dishes.

Benefits of Applying SCAMPER

- Encourages creative thinking.
- Improves user experience.
- Increases customer satisfaction.
- Enhances app functionality.
- Creates a competitive advantage.
- Helps identify innovative business opportunities.

Many ideas

Idea Generation

- AI-based meal recommendations
- Voice search
- Scheduled delivery
- One-click reorder
- Live GPS tracking
- Smart offers based on preferences

Evaluate the ideas

Idea	Feasible	User Value
AI Recommendations	High	High
Voice Ordering	Medium	High
Live Tracking	High	High
Drone Delivery	Low	High

Selected ideas

- AI recommendations
- Live order tracking
- Voice search
- One-click reorder

SCAMPER

Campus visitor Application:

- Application to facilitate the visitors

Step	Goal	Version 1	Version 2
Swap	Identify parts of the system that can be replaced with better, cheaper, faster, or more modern alternatives.		
Combine	Create new value by joining two unrelated features, services, or processes into a single unified offering		

SCAMPER

Campus visitor Application:

- Application to facilitate the visitors

Step	Goal	Version 1	Version 2
Adapt	Look outside your immediate industry or context to adapt proven solutions.		
Modify	Tweak the size, frequency, emphasis, or performance of specific features.		

SCAMPER

Campus visitor Application:

- Application to facilitate the visitors

Step	Goal	Version 1	Version 2
Put to another Use	Find secondary utility for waste, unused data, or existing infrastructure.		
Eliminate	Identify and strip away friction, unnecessary features, or redundant steps.		
Reverse	Flip the sequence, layout, or operational logic backwards to discover non-obvious improvements.		

SCAMPER

Application: Online Food ordering

Provide a new version of application

HOW will you use SCAMPER mentioning

- Changes in functionality
- Appearance
- Performance
- Etc

to enhance the application

SCAMPER

Application: Online Food ordering

Provide a new version of application

Step	Version 1(Existing)	Version 2(Suggested)
Swap		
Combine		
Adapt		
Modify		
Put to another use		
Eliminate		
Reverse		

Crazy 8's Technique in Design Thinking

Crazy 8's is a **rapid ideation technique** used during the **Ideate** phase of Design Thinking. It encourages participants to generate **eight different ideas in eight minutes** by quickly sketching solutions. The goal is to explore many possibilities rather than focusing on creating perfect designs.

Crazy: Encourages bold, creative, and unconventional ideas.

8's: Participants create **8 different sketches**.

8 Minutes: Usually, one minute is allocated for each sketch.

The technique emphasizes **speed, creativity, and quantity over perfection**.

Objectives

- Generate many ideas in a short time.
- Encourage creative thinking.
- Avoid getting stuck on the first idea.
- Explore different approaches to solving a problem.
- Promote collaboration in teams.

Materials Required

- A blank A4 sheet of paper
- Pencil or pen
- Timer (8 minutes)

Step 1: Define the Problem
Clearly state the problem or design challenge.

Example:

Design a better food ordering experience for a university cafeteria.

Step 2 : Take an A4 sheet and fold it into **8 equal sections** (2 × 4 grid)

Step 3

Set the timer for 8 minutes

Step 4: Sketch One Idea Per Box

Draw **one unique idea** in each section. The sketches should be simple and quick—stick figures, icons, and arrows are enough.

Step 5: Share and Discuss

Participants present their sketches to the group. Others can ask questions or suggest improvements.

Step 6: Select the Best Ideas

- Evaluate the ideas based on:
- User needs
- Feasibility
- Innovation
- Cost
- Ease of implementation

Sketch	Idea
1	Voice-based food ordering
2	AI meal recommendations
3	One-click reorder
4	QR code menu scanning
5	Live kitchen preparation status
6	Group ordering with friends
7	Meal subscription plans
8	AR view of food before ordering

Advantages

- Generates many ideas quickly.
- Encourages creative and innovative thinking.
- Prevents overthinking.
- Promotes equal participation in team discussions.
- Helps identify multiple solution paths before choosing one.

Limitations

- Time pressure may be challenging for some participants.
- Requires basic sketching ability (although artistic skill is not important).
- Some ideas may be unrealistic and need refinement later.

Key Takeaways

- Crazy 8's is a **fast-paced ideation technique** that helps teams think beyond their first solution.
- It promotes **divergent thinking** by encouraging multiple ideas before evaluating them.
- The emphasis is on **speed, creativity, and exploration**, making it an effective tool for generating innovative software design concepts.

What is Six Thinking Hats?

- **Six Thinking Hats** is a **creative thinking and decision-making technique** developed by **Edward de Bono**. It helps individuals and teams examine a problem from **six different perspectives**, leading to more balanced and well-informed decisions.
- Each "hat" represents a specific way of thinking. Instead of everyone arguing from different viewpoints simultaneously, the team focuses on **one type of thinking at a time**.

Instead of everyone thinking in different directions at the same time, all participants "wear" the **same colored hat** together.

Each hat represents a specific mode of thinking, allowing the team to focus on one perspective at a time.

This approach encourages balanced discussions, reduces conflict, and leads to more innovative and well-rounded solutions.

Why Use Six Thinking Hats?

- Encourages creative thinking
- Improves teamwork and collaboration
- Reduces conflict during discussions
- Ensures all perspectives are considered
- Helps make better design decisions
- Generates innovative software solutions

Hat	Purpose	Focus
White Hat	Facts and information	What do we know? What data is needed?
Red Hat	Emotions and intuition	How do users feel? What are our instincts?
Black Hat	Risks and caution	What could go wrong? What are the limitations?
Yellow Hat	Benefits and optimism	What are the advantages? Why will it work?
Green Hat	Creativity and new ideas	What innovative solutions can we generate?
Blue Hat	Process and control	What is our plan? What should we do next?

Example

Problem: Design a Smart Food Delivery App.

- **White Hat:** Users wait an average of 40 minutes for delivery.
- **Red Hat:** Users feel frustrated when delivery is delayed.
- **Black Hat:** Heavy traffic may cause late deliveries.
- **Yellow Hat:** Live order tracking increases customer satisfaction.
- **Green Hat:** Introduce AI-based route optimization and voice ordering.
- **Blue Hat:** Select the best ideas, create a prototype, and test it with users.

Advantages

- Encourages structured thinking.
- Prevents one-sided decision-making.
- Promotes teamwork.
- Improves creativity.
- Helps identify risks early.
- Leads to better software design decisions.

Limitations

Requires practice to use effectively.

Can take more time if the discussion is not well managed.

Works best with active participation from all team members.

3. Mind Mapping

- **Mind mapping is a visual technique for organizing information and exploring relationships between ideas around a central topic.**

- Instead of generating ideas randomly, students organize them into **branches and sub-branches**.

- Mind mapping is a visual brainstorming tool used in design thinking to connect thoughts, structure complex data, and spark creative solutions. It places a core problem or idea in the center and expands outward into categorized branches and sub-branches of words, themes, and user insights

Mind Mapping

- Mind Mapping: generating insights from exploration activities and using those to create design criteria.
- Mind mapping is the term use for the process of looking for patterns in the large quantity of data you've collected during your exploration of What is.
- The goal is to establish the criteria for the What if idea generation stage, which comes next.
- Describes the process of extracting meaning from a vast amount of information

Getting Started with Mind Mapping

1. Hold a yard sale. Mind mapping begins with laying out the data you've collected for everybody to see.

2. Invite shoppers. Tell a group of thoughtful people—anywhere from ten to 50—that you want to borrow their intuition for a day or even just an afternoon.

3. Offer tours. Kick off the event by asking your guests to tour the yard sale (or art gallery). Depending on their familiarity with the process, you may need to spend some time (but not too much) moving around the room explaining briefly what each visual describes.

4. Pick out the good stuff. Ask attendees to browse individually (without talking to one another) and note (on separate medium-sized Post-it notes) any learnings that he or she believes should inform new ideas.

Getting Started with Mind Mapping

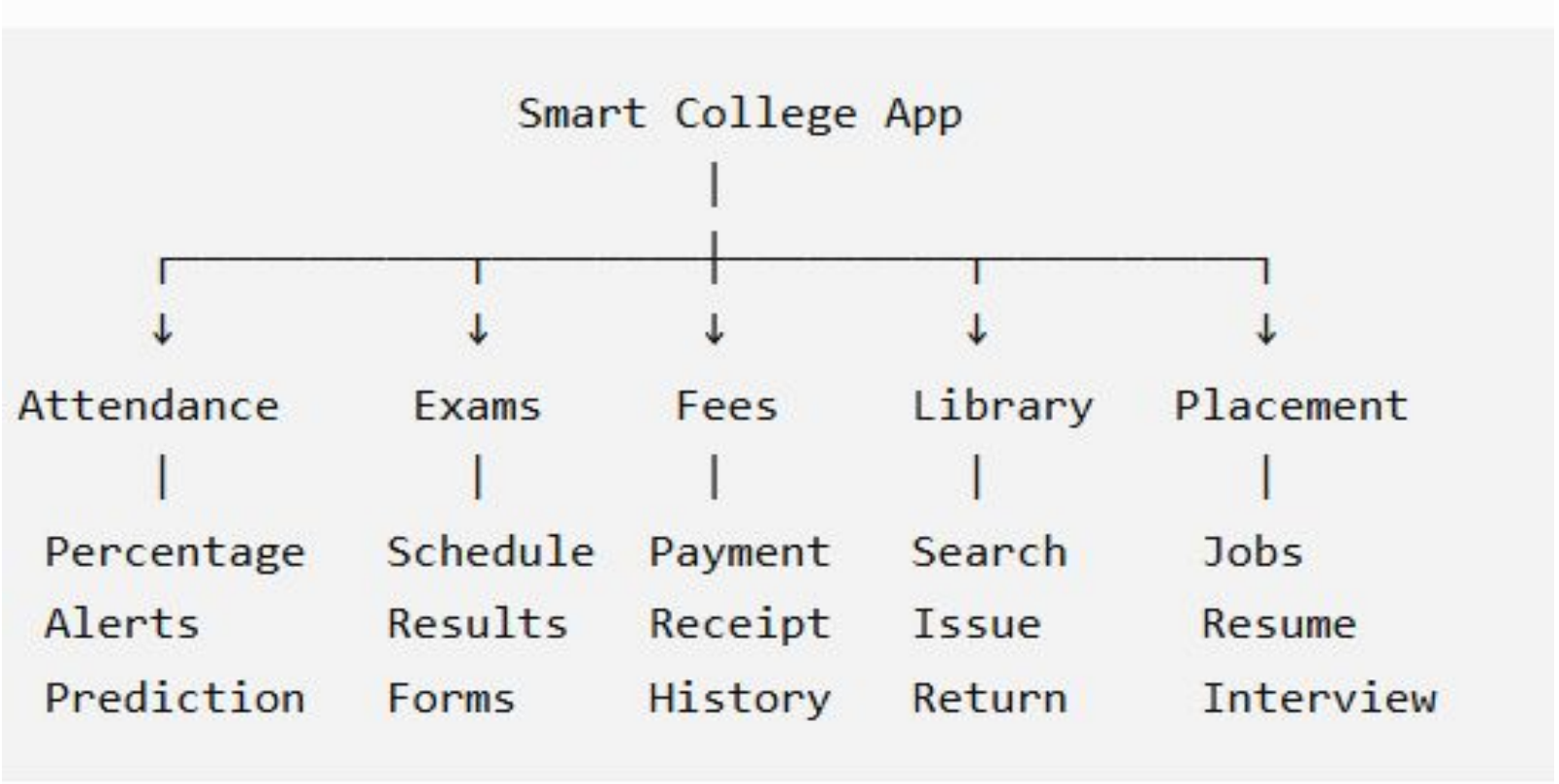
5. Cluster the good stuff. Have attendees return to their circles and spend five minutes privately sorting through their Post-its and clustering them into themes, using their clipboards.

- Team members then add to that cluster their own relevant notes.
- At the conclusion, any Post-it notes that have not been assigned to a theme are posted in the margins as “outliers.”
- Pay attention to these: Just because a piece of information doesn't fit into a cluster doesn't mean that it isn't important.

6. Identify the insights related to each cluster and look for connections between clusters.

7. Translate insights and connections into design criteria. Pose the question, “Based on what you have learned, if anything were possible our design would...”

Mind mapping




G2.com
Mind Mapping Example
Ideally 1,500+ words
Great for lead-generations
Most shared
Blog posts
Statistical infographics
Webinars
Highly trusted
Content Marketing Ideas
Accessible
Podcasts
User-generated content
Great for lead-generations
White papers
Engaging
Social media posts
Video
Easy to repurpose
Most engaging

Mind Mapping vs Brainstorming

Brainstorming	Mind Mapping
Mainly generates ideas	Organizes and connects ideas
Can be individual or group	Usually visual/individual or group
Focuses on quantity	Focuses on relationships
Ideas may initially be unstructured	Ideas are arranged hierarchically
Example: Generate 20 app ideas	Organize app features into categories

Reference:

https://www.g2.com/articles/design-thinking

- Creativity techniques: brainstorming, SCAMPER, mind mapping
- Generating multiple design alternatives
- Storyboarding and user journey mapping
- Design heuristics for software solutions
- Evaluating idea feasibility and prioritization

3.2 Generating Design Alternatives

Meaning of Alternative

- A choice limited to one of two or more possibilities, as of things, propositions, or courses of action, the selection of which precludes any other possibility.
- One of the things, propositions, or courses of action that can be chosen.
- A possible or remaining course or choice.

Design Alternatives

- When a product is available in different **designs** where all the **designs** share the same components, but the arrangement of those components is different in different **designs**, they are referred as **design alternatives**.

(Courtesy : https://www.igi-global.com)

Design Space

- An imaginary intellectual region of design alternatives.
- Contains all the potential solutions to our design problem.
- Conveys a feel for the problem at hand.
- Large design space suggests a design domain with a large number of acceptable designs.

Design Space

- Large design space may also suggest a design problem with large number of design variables.
- Morphological chart as a formal tool to generate/define design spaces.
- Morphological chart to populate designs that perform the functions we specify.
- Analogical thinking - another approach for generating design alternatives.

Morphological Chart

- A matrix in which the leftmost column is a list of all principal functions that the design must perform and some key features it must have.
- List should be of a manageable size.
- All entries should be at the same level of detail to help ensure consistency.
- List the different means of realizing the function or feature.

Morphological Chart

MEANS / FUNCTION	1	2	3	4	5	6

Morphological Chart

- Separate functions from key features.
- Morph chart can quickly become large.
- Avoid losing track of or confusing functions with key features.
- May create two “design space” models in two separate charts.
- Design must be fully functional to satisfy client’s requirements.
- Address all functions on the morph chart.

Morphological Chart

- Small number of means suggests that:
  - we have a small design space (limited choices).
  - we have not fully explored the available design space.
- Every function, listed in the leftmost column must be achieved by the design.

Morphological Chart

- Assemble designs by choosing one means from each row, combine them into a functional design concept or scheme.
- Functions listed in the leftmost column.
- Means by which each function can be implemented arrayed along a row to each entry's right and shaded.
- Some functions may have more means than others.
- Helps compute the number of potential designs.

Morphological Chart - Juice Container Design

MEANS / FUNCTION	1	2	3	4	5	6
Contain Liquid	Can	Bottle	Bag	Box	••••	••••
Fill and Seal Container	Fill and Heat Seal	Sealed Cap	Glue Container Material	Twist Top	Bottle Cap	
Empty Container	Pull Tab	Inserted Straw	Twist Top	Tear Corner	Unfold Container	Zipper
Resist Forces	Thick Walls	Flexible Materials				
Identify Product	Shape of Container	Distinctive Label	Color	••••	••••	••••

Feasible Design Alternatives - Juice Container

MEANS / FUNCTIONS	1	2	3	4	5	6
Contain Liquid	Can	Bottle	Bag	Box	••••	••••
Fill and Seal Container	Fill and Heat Seal	Sealed Cap	Glue Container Material	Twist Top	Bottle Cap	
Empty Container	Pull Tab	Inserted Straw	Twist Top	Tear Corner	Unfold Container	Zipper
Resist Forces	Thick Walls	Flexible Materials				
Identify Product	Shape of Container	Distinctive Label	Color	••••	••••	••••

Infeasible Design Combinations - Juice Container

MEANS / FUNCTIONS	1	2	3	4	5	6
Contain Liquid	Can	Bottle	Bag	Box	••••	••••
Fill and Seal Container	Fill and Heat Seal	Sealed Cap	Glue Container Material	Twist Top	Bottle Cap	
Empty Container	Pull Tab	Inserted Straw	Twist Top	Tear Corner	Unfold Container	Zipper
Resist Forces	Thick Walls	Flexible Materials				
Identify Product	Shape of Container	Distinctive Label	Color	••••	••••	••••

Morphological Chart

- Provides both a tool to develop a design space and create design alternatives.
- Provides an approach to prune that design space by identifying and excluding infeasible, incompatible alternatives.
- Includes key features as well as functions, not all combinations are feasible designs.
- Apply interface constraints, physical principles and plain common sense to exclude infeasible alternatives.

Morphological Chart

- List functions and features at the same level of detail.
- Do not develop highly detailed designs at the conceptual stage.
- Morph charts to expand design space for large, complex systems.
- List the principal subsystems in a starting column and then identify various means of implementing those subsystems.
- Can create morph charts for subsystems.

Example: Designing a Login Feature

Auth Method	Email + Password	**OAuth (Google/GitHub)**	Biometric	Magic Link
Session Handling	**JWT (stateless)**	Server session + cookie	Token in localStorage	
UI Pattern	**Modal overlay**	Full-page form	Inline embedded	
Password Recovery	**Email reset link**	SMS OTP	Security questions	

Highlighted = OAuth + JWT + Modal + Email reset link

Morphological Chart: User Authentication System

Sub-Function / Feature	Option A (Simple)	Option B (Standard)	Option C (Advanced / Secure)
1. Primary Identifier	Username / Email	Phone Number	Hardware Security Key (FIDO2)
2. Verification Method	Static Password	6-digit SMS OTP	Biometric (Face ID / Fingerprint)
3. Session State Storage	Client-side Cookies	Server-side Redis Cache	Distributed JWT (JSON Web Tokens)
4. Login UI Trigger	Standard Page Redirect	Pop-up Modal Dialog	Magic Link / Passwordless Inline Form
5. Failure Handling	Simple Error Text	Captcha + Account Lockout	Rate Limiting + Risk-based IP Challenge

Use of Metaphors

- A figure of speech used to give depth or color to the description of an object or process by likening it to more familiar, object or process.
- Points out analogies between two different situations.
- Suggest that there are parallels/similarities in two sets of circumstances.

Analogies

- Very powerful tools in engineering design.
- Direct analogy looks for parallels or similarities.
- Symbolic analogy draws connections through some underlying symbolism.
- Fantasy analogy imagines something that is literally fantastic or beyond belief - *thinking outside of the box*.

Common Software Analogies

- **Shopping Cart:** E-commerce checkout models mimic physical retail carts.
- **Physical Trash Can:** File deletion mimics throwing paper in a bin.
- **Material Design:** Google mapped digital layers to physical paper and ink responding to light and touch.

Metaphorical Solutions

- Similar solutions - can look for similar ideas.
- Contrasting solutions – look for opposite ideas.
- Contiguous solutions - thinking of adjoining (or adjacent) ideas, takes advantage of natural connections between ideas, concepts, and artifacts.

Example:

1. A classic example of a metaphorical design in software is the **Desktop Metaphor**, which maps a computer screen to a physical office desk. Files act as paper documents, folders hold those documents, and a trash can represents deletion. This helps users understand abstract computing functions instantly.

Metaphorical Thinking

- Divergent thinking :

  - try to remove limits or barriers, hope to increase the store of design ideas and choices.

  - “think outside of the box”.

- Convergent thinking :

  - narrow our design space, focus on best alternative(s).

  - “stay within our game”.

Metaphorical Thinking

- Hybrid thinking :
  - Mix divergent and convergent thinking.
  - Think outside the box, but stay within the game!
- Metaphorical thinking is also related to a characterization of design.

6 – 3 - 5 Method

- Design done in team setting.
- Six team members seated around a table participate in an idea generation “game,” .
- Each of them writes down three design ideas in key words and phrases.
- Six individual lists are then circulated past each of the remaining team members in a sequence of five rotations of written (only) comment and annotation.

6 – 3 - 5 Method

- Verbal communication or cross talk not allowed.
- Each list makes a complete circuit around the table.
- Each member of the team stimulated by increasingly annotated lists of the other team members.
- All the participants to comment on each of the lists.

6 – 3 - 5 Method

- The team lists, discusses, evaluates, and records all the design ideas that have resulted from a group enhancement of the individual team members' ideas in a common visualization medium (projector, black board etc.).
- m team members use (m – 1) rotations to complete a cycle : m – 3 – (m – 1) method.
- Natural upper limit of m = 6 , prefer fewer.

The 6-3-5 Method: Brainwriting

Six team members, three ideas each, five silent rotations — keeps every voice contributing without cross-talk.

1

Write

6 engineers each write 3 ideas.

2

Pass

Sheets rotate — no talking, only written notes.

3

Build

Teammates extend the ideas in front of them.

4

Converge

After 5 rotations, the team reviews everything.

C - Sketch Method

- Sketching as a natural form of thinking.
- A team seated around a table.
- Each member sketches one design idea on a piece of paper.
- Proceeds further as does the 6 – 3 – 5 method.
- Only permissible communication is by pencil on paper.

C - Sketch Method

- Appealing in areas like mechanical design.
- Drawings and diagrams facilitate the grouping of relevant information.
- Helps people to better visualize the objects being discussed.

Gallery Method

- Team members first develop individual, initial ideas within the allotted time.
- All resulting sketches posted on a corkboard/conference room whiteboard.
- Set of sketches serves as the backdrop for an open, group discussion.
- Questions asked, critiques offered, and suggestions made.

Gallery Method

- Each participant returns to his/her drawing, suitably modifies or revises it within a specified period of time.
- Produces a second-generation idea.
- Iterative and progressive.
- Can't predict how many cycles of individual idea generation and group discussion required.

Gallery Method

- Proceed until a consensus emerges within the group.
- Quit when one more cycle will not gain any new information, when a saturation plateau is reached.

Design Generation

- An exciting goal - directed creative activity.
- Designed to serve a known purpose, not to search for one.
- Goal may be imposed externally or internally.
- There is a goal towards which the creative activity is aimed.
- Creative activity requires work.

Design Generation

- Beyond the morph chart, beyond the team-based tools, what else can be done to generate design ideas?
- How to usefully navigate, expand or, if needed, contract the design space?
- Will have to design more subsystems and components.
- Will have to combine and connect the subsidiary individual designs.

Navigating Design Spaces

- Large design spaces are complex.
- Hundreds or thousands of design variables must be assigned.
- Design spaces are complex because of interactions between subsystems and components.
- Collaboration with many specialists critical.
- Divide and conquer.
- Contract design spaces when too large.

Navigating Design Spaces

- For a small or bounded design space, the number of potential designs is limited or small, or the number of design variables is small and they take on values within limited ranges.
- Design of individual components of large systems occurs within small design spaces.
- Expand design spaces when too small.

Expanding Design Spaces

- Conduct literature reviews, determine the state of the art and identify prior work in the field.
- Conduct a patent search, identify available technologies, do not reinvent the wheel.
- Benchmark existing products to evaluate how well they perform.
- Reverse engineering to see how functions are performed, identify alternate means.

Contracting Design Spaces

- Prune or contract design space to make it more manageable.
- Check for external constraints that affect the design.
- Invoke and apply constraints while assessing the presence of external constraints.
- Freeze the number of features and behaviors being considered, avoid details that are unlikely to seriously affect the design.

Contracting Design Spaces

- Impose some order on the list, hark back to data gathered during problem definition.
- Some particular functions or features may be more important.
- “Get real!”; apply common sense to rule out infeasible ideas.

Know When to Expand or Contract



Expand

When the space feels too small or too few options surface — for a single microservice, a form rule, a caching layer.



Contract

When the space is too large or complex — a platform migration, a multi-service architecture with dozens of variables.

Expanding: Widen Your Options

- Prior Art & Literature — read RFCs and engineering blogs.
- Open-Source Search — reuse existing libraries and frameworks.
- Competitor Benchmarking — try competing products directly.
- Reverse Engineering — inspect how existing APIs achieve a function.

Contracting: Prune Your Options

- Apply Constraints — filter by latency, scale, security, cost.
- Freeze Scope Early — defer nice-to-haves for this iteration.
- Prioritize by Impact — rank options against real requirements.
- "Get Real" Check — apply plain engineering judgment.

Final Word

- Sea of Alternatives at hand.
- Best Alternative?
- Blend possibilities :
  - morph charts.
  - design generation games.
  - information gained from research.
  - experience-based judgments.
  - gut-level feelings.

Final Word

- Think strategically :
  - group and reorganize the functions and the resultant design alternatives.
- Never mind all the alternatives.
- Pick the winner.
- Resources (time, money, man power etc.) are rare.

Design heuristics for software solutions

Design heuristics are practical **rules of thumb** that help a designer make good software-design decisions when there is no single mathematically correct solution.

In **Software Design Thinking**, heuristics are especially useful during the transition from **problem understanding** → **ideation** → **conceptual design** → **detailed design**, because they help teams evaluate and improve possible solutions

What are Design Heuristics?
A **heuristic** is a general principle that guides design decisions.

For example:
“Keep the user interface simple and consistent.”

Instead of asking only *“Can we implement this feature?”*, a designer asks:

Is it easy for the user?
Is the solution understandable?
Can it handle future changes?
Is the system divided into meaningful components?
Is unnecessary complexity avoided?
Does the design satisfy the requirements?

Why Do We Need Design Heuristics?

Design heuristics help designers:

Purpose	Explanation
Reduce complexity	Avoid unnecessarily complicated solutions
Improve usability	Make systems easier to learn and use
Improve maintainability	Make future modifications easier
Improve reliability	Reduce opportunities for errors
Support decision making	Compare alternative designs
Encourage reuse	Use existing components/patterns
Improve scalability	Prepare the system for growth
Improve consistency	Similar functions should behave similarly

Heuristics Design Principles by Nielsen

Usability Heuristics



Visibility Of System Status



Match Between System & Real World



User Control And Freedom



Consistency And Standards



Error Prevention



Recognition Rather Than Recall



Flexibility And Efficiency Of Use



Aesthehic And Minimalist Design



Help Users With Errors



Help And Documentation

10 Usability Heuristics

1: Visibility of System Status
2: Match Between the System and the Real World
3: User Control and Freedom
4: Consistency and Standards
5: Error Prevention
6: Recognition Rather than Recall
7: Flexibility and Efficiency of Use
8: Aesthetic and Minimalist Design
9: Help Users Recognize, Diagnose, and Recover from Errors
10: Help and Documentation

Visibility of System Status

- The design should always keep users informed about what is going on, through appropriate feedback within a reasonable amount of time.
- When users know the current system status, they learn the outcome of their prior interactions and determine next steps.
- Predictable interactions create trust in the product as well as the brand.

Visibility of System Status



Example of Usability Heuristic #1:
You Are Here indicators on mall maps show people where they currently are, to help them understand where to go next.

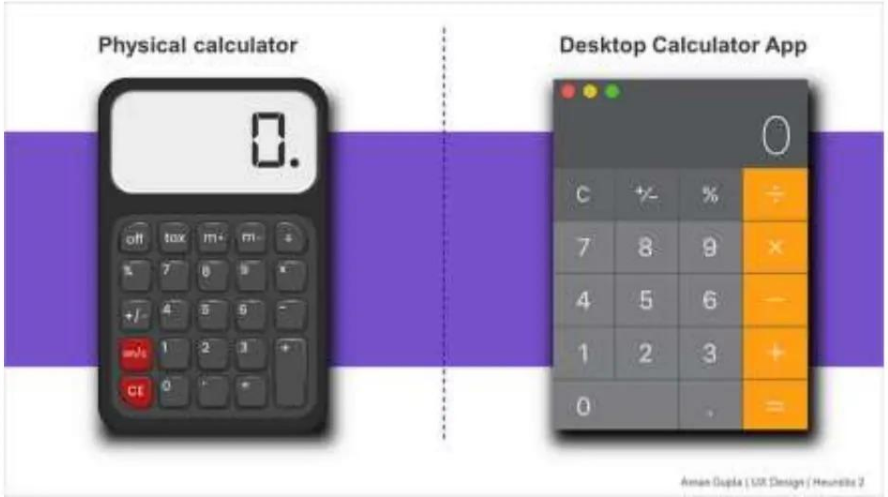
- Communicate clearly to users what the system's state is — no action with consequences to users should be taken without informing them.
- Present feedback to the user as quickly as possible (ideally, immediately).
- Build trust through open and continuous communication.

Match between system and the real world

- The system should speak the users' language, with words, phrases and concepts familiar to the user, rather than system-oriented terms.
- Follow real-world conventions, making information appear in a natural and logical order
- The way you should design depends very much on your specific users.
- Terms, concepts, icons, and images that seem perfectly clear to you and your colleagues may be unfamiliar or confusing to your users.

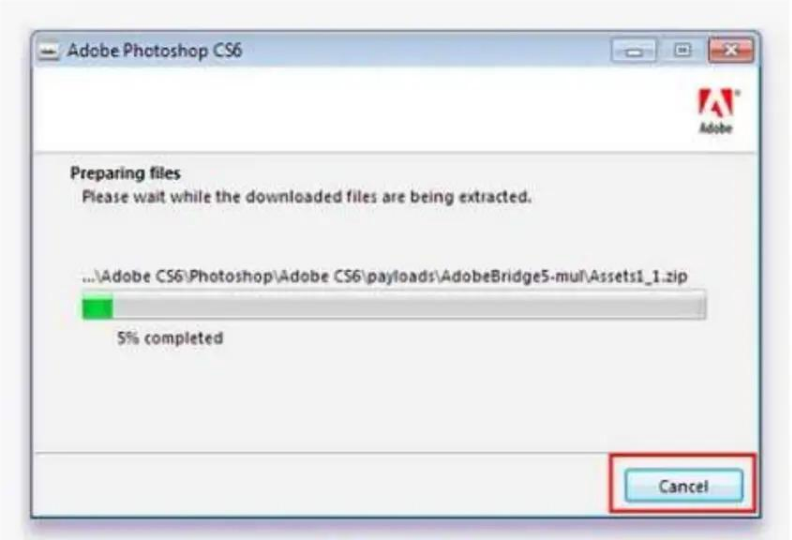
Match between system and the real world

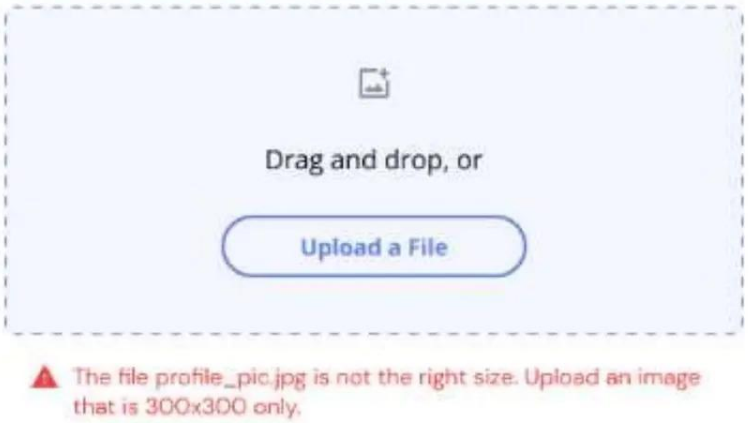
- Ensure that users can understand meaning without having to go look up a word’s definition.
- Never assume your understanding of words or concepts will match that of your users.
- User research will uncover your users' familiar terminology, as well as their mental models around important concepts.



User control and freedom

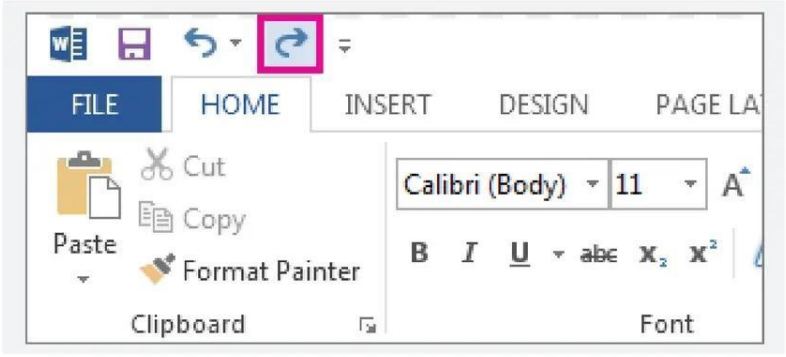
- Users often choose system functions by mistake and will need a clearly marked "emergency exit" to leave the unwanted state without having to go through an extended dialogue.
- Support undo and redo.





User control and freedom

- Support *Undo* and *Redo*.
- Show a clear way to exit the current interaction, like a Cancel button.
- Make sure the exit is clearly labeled and discoverable.



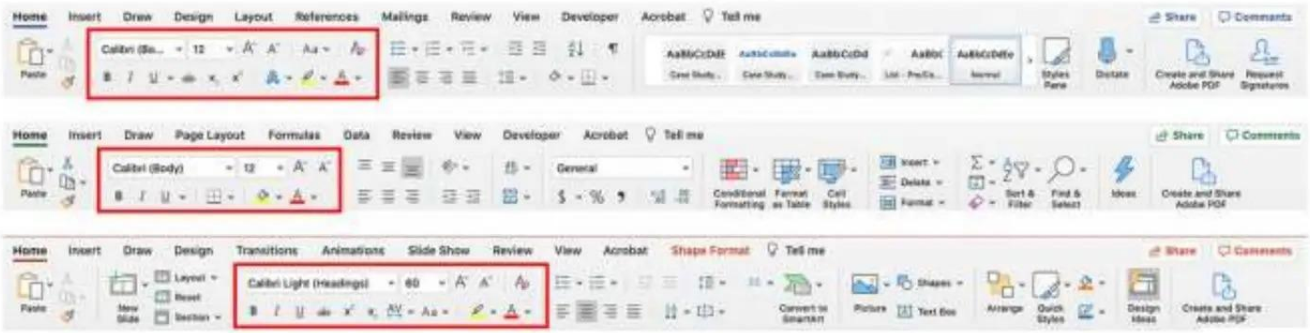


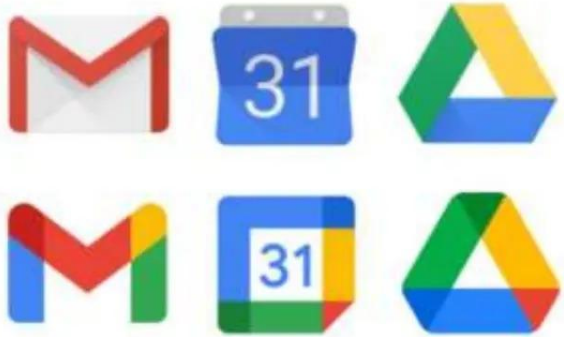
Consistency and standard

- Users should not have to wonder whether different words, situations, or actions mean the same thing. Follow platform conventions.
- Failing to maintain consistency may increase the users' cognitive load by forcing them to learn something new.
- Improve learnability by maintaining both types of consistency: internal and external.

Consistency and standard

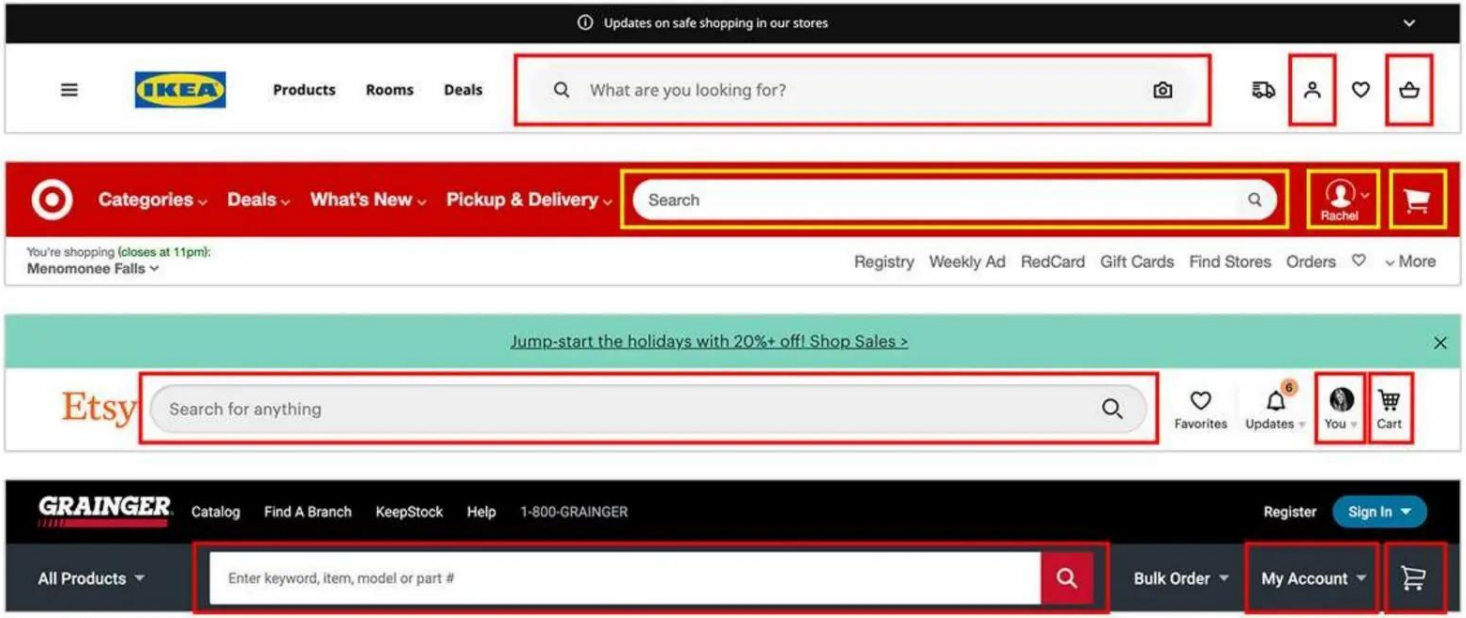
- Internal consistency relates to consistency within a product or a family of products, either within a single application or across a family or suite of applications.





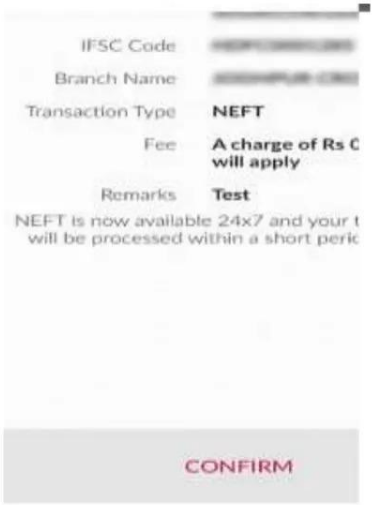
Consistency and standard

- External consistency refers to established conventions in an industry or on the web at large, beyond one application or family of applications.

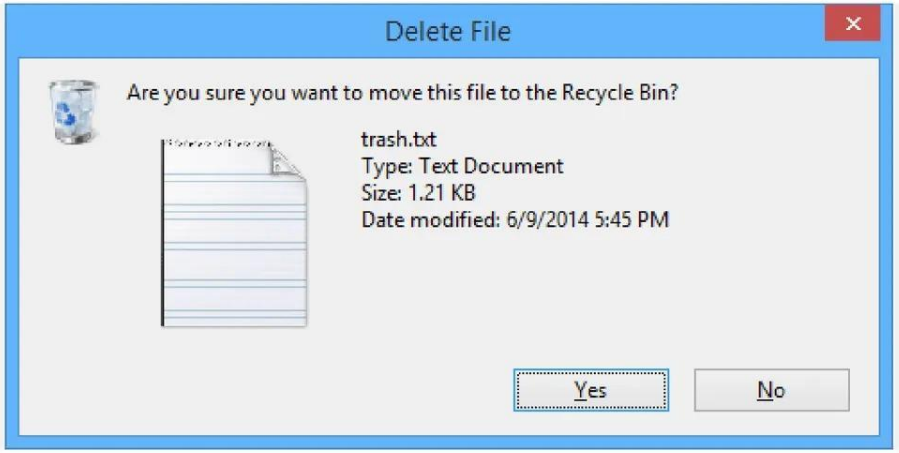


Error prevention

- minimize the likelihood of making mistakes
- Good error messages are important, but the best designs carefully prevent problems from occurring in the first place.
- Either eliminate error-prone conditions, or check for them and present users with a confirmation option before they commit to the action.

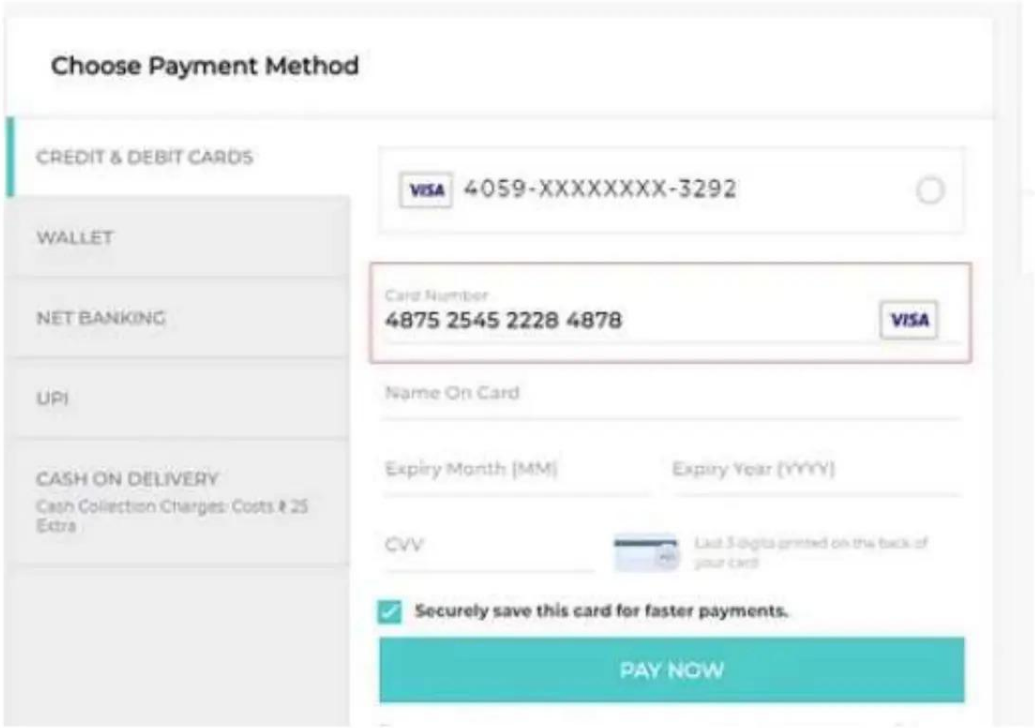


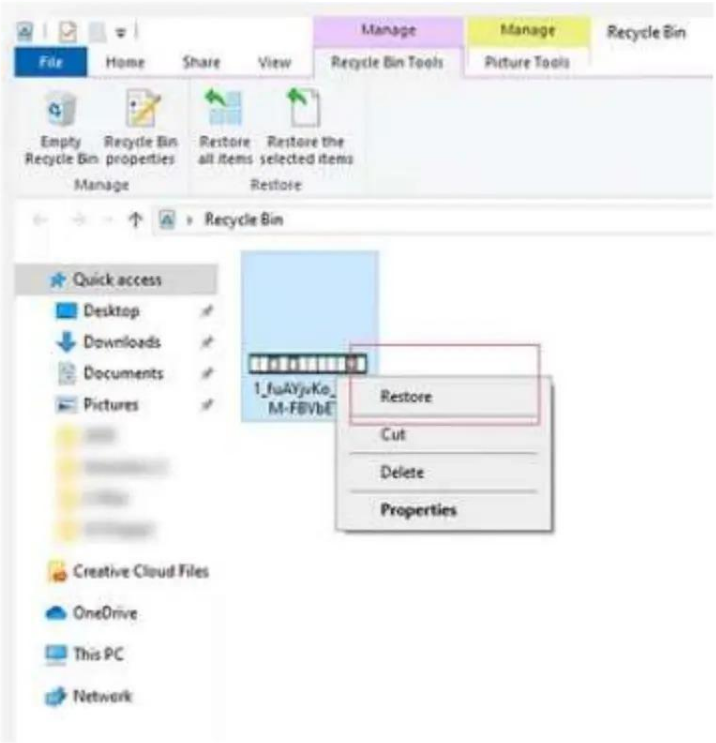




Error prevention

- Prioritize your effort: Prevent high-cost errors first, then little frustrations.
- Avoid slips by providing helpful constraints and good defaults.
- Prevent mistakes by removing memory burdens, supporting undo, and warning your users.

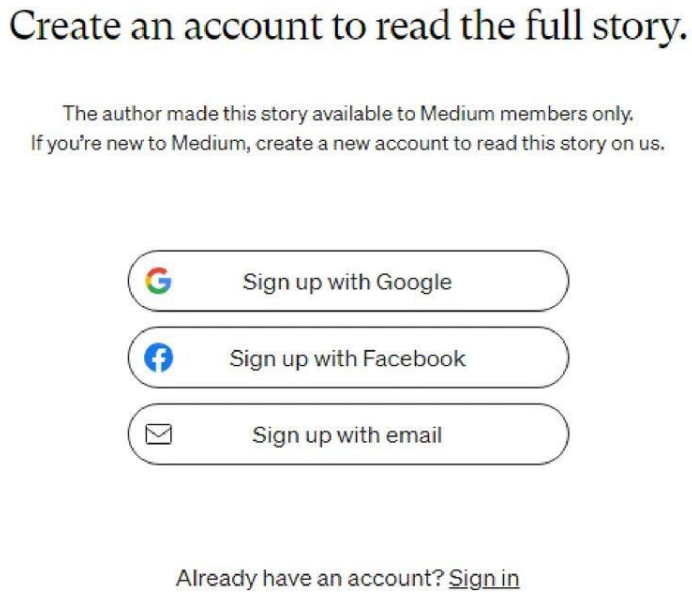




Recognition rather than recall

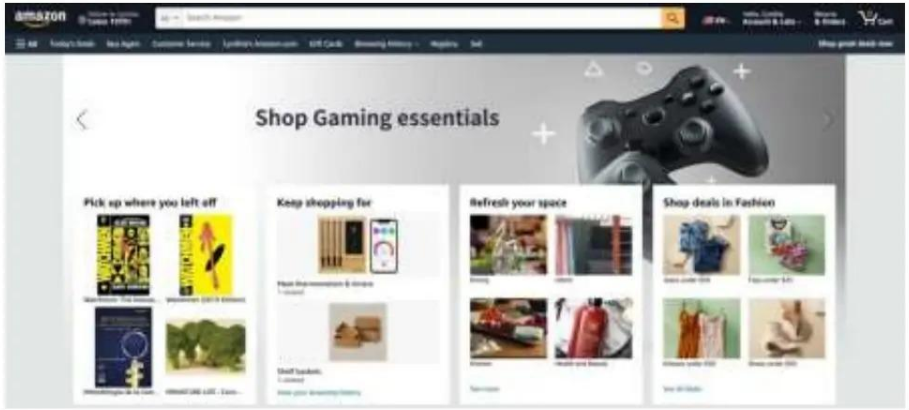
- Minimize the user's memory load by making elements, actions, and options visible.
- The user should not have to remember information from one part of the interface to another.
- Information required to use the design (e.g. field labels or menu items) should be visible or easily retrievable when needed.

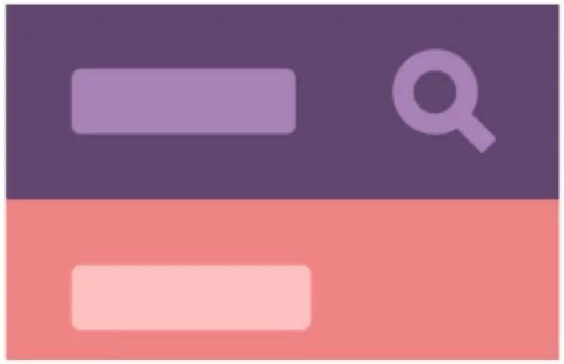


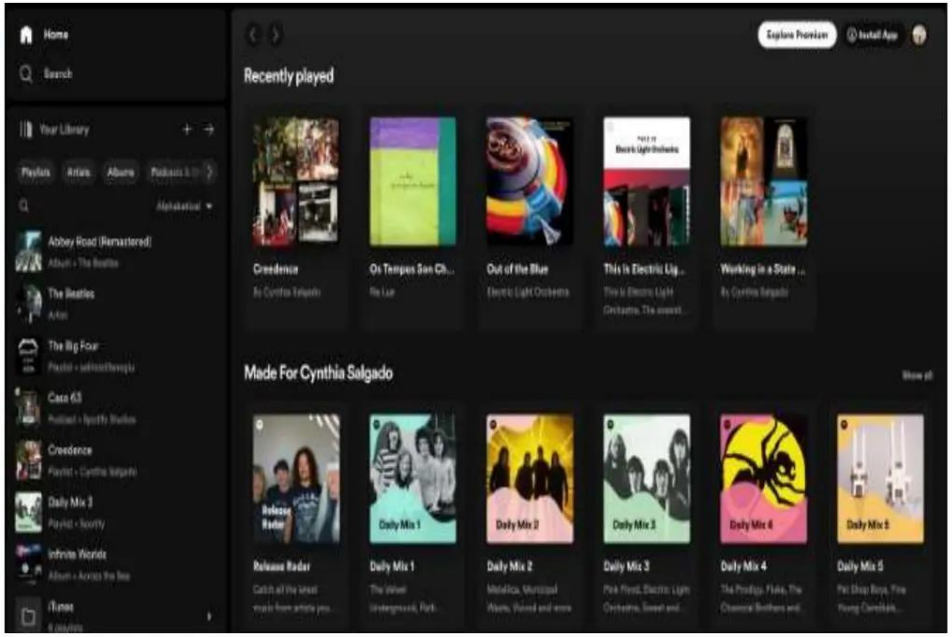


Recognition rather than recall

- Let people recognize information in the interface, rather than forcing them to remember (“recall”) it.
- Offer help in context, instead of giving users a long tutorial to memorize.
- Reduce the information that users have to remember.

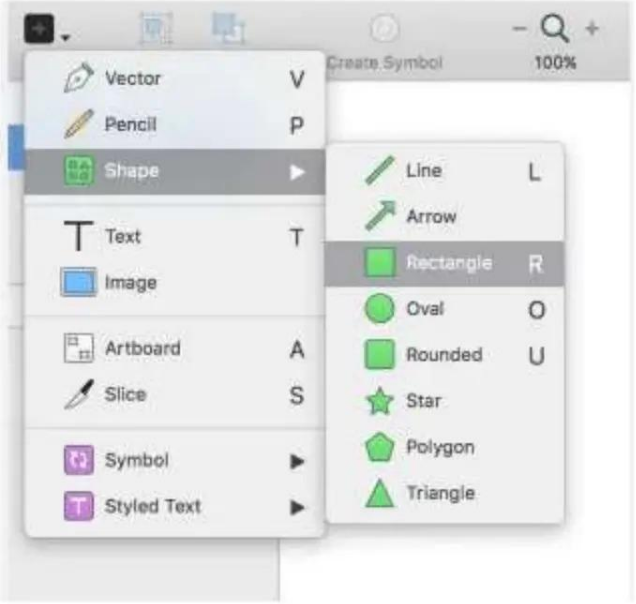


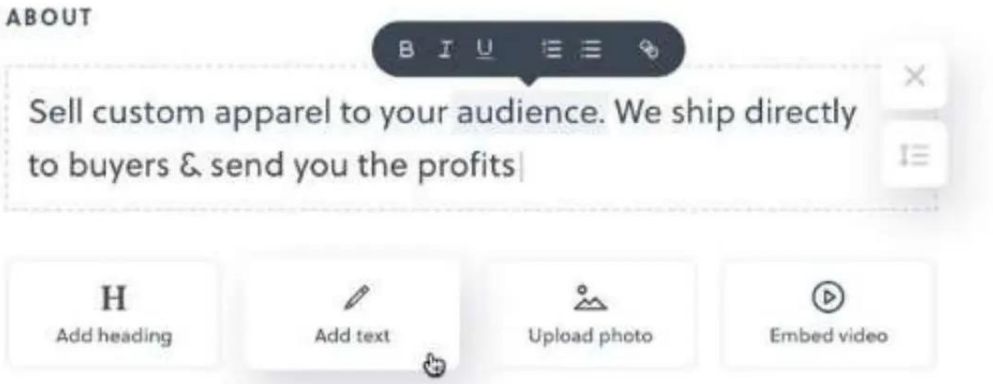




Flexibility and efficiency of use

- Shortcuts — hidden from novice users — may speed up the interaction for the expert user so that the design can cater to both inexperienced and experienced users. Allow users to tailor frequent actions.
- Flexible processes can be carried out in different ways, so that people can pick whichever method works for them.





Flexibility and efficiency of use

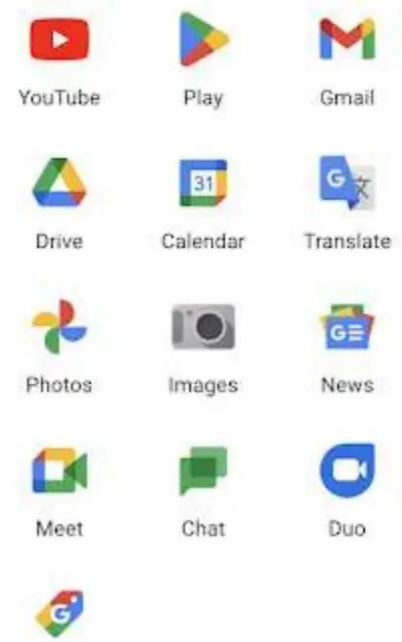
- Provide accelerators like keyboard shortcuts and touch gestures.
- Provide personalization by tailoring content and functionality for individual users.
- Allow for customization, so users can make selections about how they want the product to work.

An aesthetic and minimalist design

- Interfaces should not contain information that is irrelevant or rarely needed.
- Every extra unit of information in an interface competes with the relevant units of information and diminishes their relative visibility.





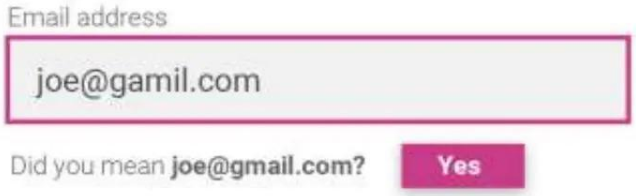


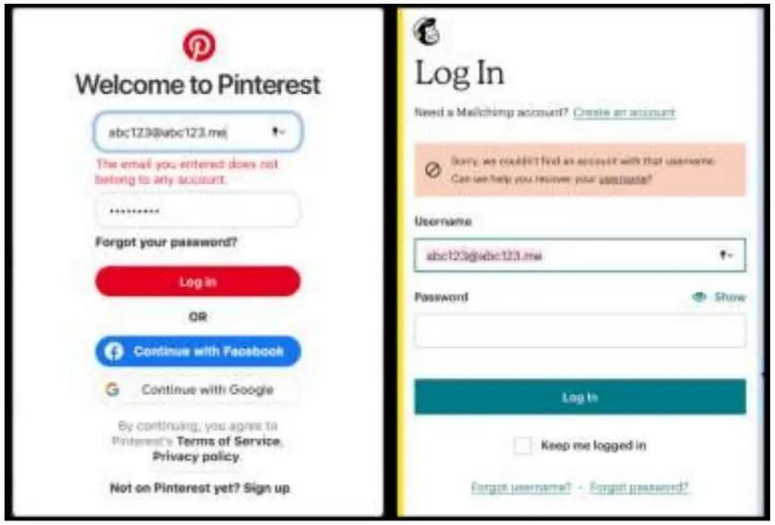
An aesthetic and minimalist design

- Keep the content and visual design of UI focused on the essentials.
- Don't let unnecessary elements distract users from the information they really need.
- Prioritize the content and features to support primary goals.

Help users recognize, diagnose, and recover from errors

- Error messages should be expressed in plain language (no error codes), precisely indicate the problem, and constructively suggest a solution.
- These error messages should also be presented with visual treatments that will help users notice and recognize them.

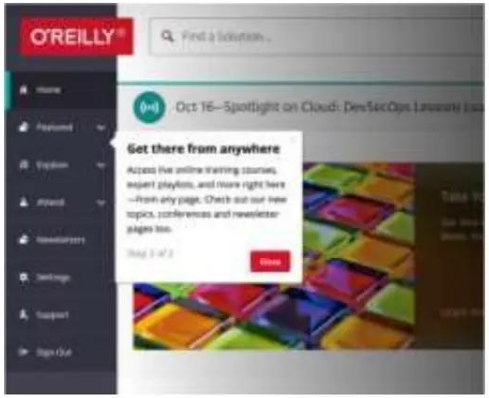




Help and documentation

- It’s best if the system doesn’t need any additional explanation. However, it may be necessary to provide documentation to help users understand how to complete their tasks.
- Help and documentation content should be easy to search and focused on the user's task. Keep it concise, and list concrete steps that need to be carried out.





10 Usability Heuristics

1: Visibility of System Status
2: Match Between the System and the Real World
3: User Control and Freedom
4: Consistency and Standards
5: Error Prevention
6: Recognition Rather than Recall
7: Flexibility and Efficiency of Use
8: Aesthetic and Minimalist Design
9: Help Users Recognize, Diagnose, and Recover from Errors
10: Help and Documentation

A Practical Heuristic Checklist

When design a software solution, ask them these questions:

User
Is the solution easy to understand?
Does it solve the actual user's problem?
Are important functions easy to find?

Functionality
Does the design satisfy all requirements?
Are error cases considered?
Is feedback provided?

Architecture
Are responsibilities separated?
Is coupling low?
Is cohesion high?
Are interfaces clearly defined?

Maintainability
Is the design modular?
Can components be modified independently?
Is unnecessary duplication avoided?

Quality
Is it secure?
Is it scalable?
Is it reliable?
Is it usable?
Is it accessible?

Evolution
What happens if requirements change?
Can new functionality be added easily?
Can existing components be reused?

Summary of Major Heuristics

Heuristic	Simple Meaning
Simplicity	Keep the solution as simple as possible
Modularity	Divide the system into manageable components
High cohesion	Keep related responsibilities together
Low coupling	Minimize dependencies between components
Separation of concerns	Give each component a clear responsibility
DRY	Avoid unnecessary duplication
Reusability	Reuse suitable components
Consistency	Similar things should behave similarly
Visibility	Make important information easy to see
Feedback	Tell users what happened
Error prevention	Prevent mistakes where possible
Abstraction	Hide unnecessary implementation details
Scalability	Prepare for growth
Security	Build security into the design
Failure handling	Expect and handle failures
Design for change	Make future modifications easier
Patterns	Reuse proven solutions
Trade-offs	Balance competing quality attributes

Problem

↓

Design Thinking

↓

Ideation

↓

Alternative Solutions

↓

Design Heuristics

↓

Evaluate Alternatives

↓

Conceptual Design

↓

Architecture

↓

Detailed Design

↓

Design Patterns

↓

Prototype

↓

Test & Iterate

Heuristics are criteria/rules used to judge whether a proposed software solution is a good design

Idea Feasibility and Prioritization

Feasibility is the technical filter applied to human-centered ideas.

Feasibility determines what engineering teams can *actually build and maintain* reliably

Desirability focuses on what users *want*

Viability focuses on what the business *should* build,

Idea Feasibility and Prioritization

Feasibility evaluates whether a proposed software solution can be

- practically built
- Deployed (in time), and
- In the budget specified
-

Within real-world technical, financial, operational, and time constraints

- Focuses directly on
  - architectural viability
  - system constraints, and
  - technical risk reduction before writing production code.

Idea Feasibility and Prioritization

Technical Feasibility Evaluates whether the system can be built using current technology, available tools, and engineering capabilities.

- **Architecture & Infrastructure:** Does the proposed design (e.g., event-driven microservices vs. monolithic) fit the existing technology stack and infrastructure?
- **Data & Integration:** Are the required APIs, data pipelines, and third-party services available, documented, and reliable?
- **Performance & Scalability:** Can the design satisfy system SLAs, such as response times, throughput (RPS), and concurrency targets?
- **Team Skills:** Does the development team possess the necessary expertise (e.g., Rust, Distributed Systems, ML pipelines), or will a steep learning curve derail timelines?

Idea Feasibility and Prioritization

Operational Feasibility Determines how well the software solution aligns with daily operational workflows and maintenance capabilities.

- **Maintainability & Technical Debt:** Will the design create an unmaintainable codebase or introduce excessive technical debt?
- **Observability & Monitoring:** Can the system be effectively monitored, logged, and debugged in production environments?
- **DevOps & Deployment:** Does the deployment architecture support CI/CD automation, rolling updates, and disaster recovery?

Idea Feasibility and Prioritization

Economic & Resource Feasibility (Cost-Benefit)
Assesses whether the lifetime cost of developing and running the software justifies its business value.

- **Cloud & Compute Costs:** What are the projected hosting, database, bandwidth, and API usage costs at scale?
- **Development Effort:** How many person-months are required to build, test, and release the system?

Idea Feasibility and Prioritization

Legal, Security & Regulatory Feasibility Ensures the technical architecture complies with statutory regulations and security standards.

- **Data Privacy:** Does the software architecture satisfy regulations like GDPR, CCPA, or HIPAA (e.g., data encryption at rest/in transit, data residency)?
- **Security Architecture:** Does the design adhere to security best practices (e.g., Zero-Trust, RBAC, secure credential storage)?
- **Techniques for Validating Software Feasibility**
- To prove feasibility before committing to full-scale development, engineering teams use several lightweight techniques:

Idea Feasibility and Prioritization

- **Technical Spikes:** Short, focused research tasks or throwaway code snippets used to explore an unknown technology or evaluate API capabilities.
- **Proof of Concept (PoC):** A small, bare-bones implementation designed to prove that a core technical concept (e.g., sub-50ms search indexing) is possible.
- **Architecture Prototyping:** Building a skeleton of the system to test non-functional requirements like load balancing, database read/write throughput, and messaging queues.
- **Risk Matrix Analysis:** Identifying high-risk design decisions (e.g., reliance on a single third-party vendor) and developing fallback architectures.

Idea Feasibility and Prioritization

1. Architectural & Stack Compatibility

- Evaluates how easily a proposed concept integrates into the current software ecosystem.
- **Concern:**
  - Can the feature leverage existing microservices and databases, or does it require a complete architectural refactor?
  - **External Dependability:** Does it introduce technical debt or lock the team into unsupported third-party vendors?

Idea Feasibility and Prioritization

2. Data Availability & Integration

- Software features are heavily constrained by data access, quality, and structure.
- **Concerns:**
  - Are internal or external APIs available to supply real-time data?
  - Is the data cleansed, structured, and compliant with privacy standards (General Data Protection Regulation (GDPR)?
  - If building AI/ML features, is there sufficient training data?

Idea Feasibility and Prioritization

3. Performance & Scalability Constraints

- Evaluates whether the system can meet non-functional requirements under real-world conditions.
- **Concern:** ,
  - Can the proposed architecture meet **Performance Service Level Agreement (PSLA)** response time, Uptime / Availability, Latency & Response Time, Error / Failure Rate, Throughput, MTTR, Scaling?

Idea Feasibility and Prioritization

4. Resource & Team Capability

- Measures whether the team possesses the skill sets, tooling, and bandwidth to build the software.
- **Concerns:**
  - Does the engineering team have experience with the required technology
  - Will it require a steep learning curve that threatens project deadlines?

Idea Feasibility and Prioritization

5. Security, Risk & Regulatory Compliance

- Identifies vulnerability exposure and regulatory blockers early in the design cycle.
- **Concerns:**
  - Does the design introduce new attack vectors (sensitive user data exposure, third-party script injection)?
  - Does it comply with industry standards
  - **Payment Card Industry Data Security Standard (PCI-DSS)**
  - Global technical and operational mandate required for any organization that stores, processes, or transmits credit cardholder data.
  - **System and Organization Controls 2 (SOC 2)**E
  - Evaluates how well a service provider manages and protects customer data, making it a critical standard for SaaS providers, cloud vendors, and technology companiesSOC2)?

Idea Prioritization

- Translates wild user-centric concepts into actionable engineering roadmaps.
- Balances human desirability with technical feasibility and business viability.

Approaches:

- **Value vs. Effort Matrix (2x2):**
- **MoSCoW Method:**
- **KANO Model:**
- **RICE Scoring**

Idea Prioritization

Value vs. Effort Matrix (2x2):

- Plots high/low user value against high/low implementation effort to yield four buckets:
- ***Quick Wins***
- High Value, Low Effort — Prioritize first.
- ***Major Projects***
- High Value, High Effort — Plan strategic roadmaps.
- ***Fill-ins***
- Low Value, Low Effort — Address during downtime.
- ***Thankless Tasks***
- Low Value, High Effort — Drop or deprioritize.

Idea Prioritization

MoSCoW Method:

Categorizes requirements into operational groups to establish clear release boundaries:

- ***Must Have*** — Non-negotiable core functionality required for launch.
- ***Should Have*** — High-value features with workaround solutions if delayed.
- ***Could Have*** — Nice-to-have additions if time and budget permit.
- ***Won't Have*** — Excluded features for the current release window.

Idea Prioritization

KANO Model

Measures how individual product features impact customer satisfaction compared to the effort or sophistication required to implement them

Basic Expectations — Table-stakes features users take for granted (e.g., login).

Performance Features — Linear features where more yields higher satisfaction (e.g., load speed).

Delighters — Unexpected, innovative features that trigger delight.

Idea Prioritization

KANO Model

Feature Categories

The model classifies product features into five distinct categories based on user perception:

1. **Must-Be (Basic / Threshold):**
   - Features that customers take for granted.
   - Their presence does not increase satisfaction, but their absence causes severe dissatisfaction.
     (Example: A login screen or working brakes on a car).

Idea Prioritization

KANO Model

Feature Categories

2. One-Dimensional (Performance):

- Features with a direct, linear relationship to satisfaction.
- The better or faster they perform, the happier the user is.
  (Example: Battery life or search execution speed).

Idea Prioritization

KANO Model

Feature Categories

3. Attractive (Delighters / Excitement): Unexpected features that spark delight. Their absence doesn't hurt satisfaction because users don't expect them, but their presence sets the product apart. *(Example: A personalized, automated workflow or free hotel room upgrade).*

4, Indifferent: Features that users simply do not care about either way. Investing time or money in them yields zero return in user satisfaction.

5, Reverse: Features that actively annoy users when present. *(Example: Aggressive pop-up ads or forced, unnecessary multi-step checkout processes).*

Idea Prioritization

KANO Model

Feature Categories

3. Attractive (Delighters / Excitement):

- Unexpected features that spark delight.
- Their absence doesn't hurt satisfaction because users don't expect them, but their presence sets the product apart.
- *Example: A personalized, automated workflow or free hotel room upgrade.*

Idea Prioritization

KANO Model

Feature Categories

4. Indifferent:

- Features that users simply do not care about either way.
- Investing time or money in them yields zero return in user satisfaction.

5. Reverse:

- Features that actively annoy users when present. *Example: Aggressive pop-up ads or forced, unnecessary multi-step checkout processes.*

Idea Prioritization

RICE Scoring:

- Uses quantitative scoring to compare features mathematically
- Designed to evaluate and rank feature ideas, design concepts, or product initiatives objectively.

$$\text{RICE Score} = \frac{\text{Reach} \times \text{Impact} \times \text{Confidence}}{\text{Effort}}$$

Idea Prioritization

RICE Scoring:

- **Reach (R):**
- How many target users will be impacted by this design idea within a specific timeframe (e.g., users per quarter)? Measured as a raw number.
- **Impact (I):**
- How significantly this feature will improve the user experience or business goal. **Standard scale:**

  3= Massive impact

  2= High impact

  1= Medium impact

  0.5= Low impact

  0.25= Minimal impact

Idea Prioritization

RICE Scoring:

- **Confidence (C):**

  How certain team is about Reach, Impact, and Effort estimates, expressed as a percentage:

  100%= High confidence (backed by quantitative data and research)

  80%= Medium confidence (supported by qualitative feedback or proxy data)

  50%= Low confidence (an educated guess or intuition)

- **Effort (E):**

  The total time required from all team members (designers, developers, QA) to ship the feature, typically measured in **person-months**.

Idea Prioritization Approach Comparison

Approach	Best Used For	Primary Strengt	Potential Pitfall
Value vs. Effort	Early-stage design workshops	Fast visual consensus	Subjective estimations
MoSCoW	Fixed-scope release planning	Clear boundaries for MVPs	Everything becomes a "Must Have"
KANO Model	User Experience (UX) refinement	Validates true user desire	High research & survey overhead
RICE Scoring	Mature product roadmap triage	Data-backed objectivity	Complex math based on estimates